

Welcome to Day 21

Moving into LangGraph Territory

- Building on previous agent frameworks (OpenAI Agents SDK, Crew AI)
- Focus: **Stability, resiliency, and repeatability**
- New way of thinking about agent workflows



KEY POINT

LangGraph offers a different approach — organizing workflows as graphs.

The LangChain Ecosystem

Three Main Products

- 1 **LangChain** - Original framework for LLM abstractions
- 2 **LangGraph** - Modern agent workflow orchestration
- 3 **LangSmith** - Monitoring and debugging tool



IMPORTANT

LangGraph is independent from LangChain (can use any framework).

 <https://www.langchain.com/langgraph>

What is LangChain?

- **Original Purpose:** Simplify API integrations
- **Evolution:** Chaining LLM calls together
- **Features:**
 - RAG (Retrieval-Augmented Generation)
 - Prompt templates
 - Memory management
 - Declarative language (LCEL)
 - Tool abstractions



NOTE

More general-purpose, predates modern agent excitement.

What is LangGraph?

🎯 Core Purpose

Platform for complex agent workflows

FOCUS AREAS

- **Stability** and fault tolerance
- **Resiliency** at scale
- **Repeatability** and monitoring
- Human-in-the-loop capabilities



KEY INSIGHT

Organizes workflows as graph structures (tree-like).

LangGraph Key Features



Agent-Driven Experiences

- Multi-agent collaboration allowing complex task distribution
- Conversation history and memory persistence across sessions
- Time travel (checkpointing) to inspect and resume states
- Human-in-the-loop interactions for approval and guidance



Deployment

- Fault-tolerant scalability for enterprise workloads
- Ability to restore the application state to any point in time for debugging or retry

 ROBUST INFRASTRUCTURE

LangGraph Components



The Three Parts

1. LangGraph Framework

Core library (our focus)

2. LangGraph Studio

Visual builder tool

3. LangGraph Platform

Hosted deployment solution



Conceptual Analogy

Understanding the ecosystem through familiar tools:

Similar to Crew AI

Structure mirrors the Framework, Studio, and Enterprise tiers.



Framework



Studio



Platform

LangSmith Integration

Separate Monitoring Tool

- Works with both **LangChain** and **LangGraph**
- **Capabilities:**
 - Visibility into LLM calls
 - Reasoning traces
 - Quick debugging of failures



IMPLEMENTATION NOTE

We'll use LangSmith to monitor our graphs

WEBSITE

<https://smith.langchain.com/>

Anthropic's Perspective

” From "Building Effective Agents" Blog Post

- Frameworks simplify low-level tasks
- **BUT:** Create abstraction layers that obscure prompts
- Can make debugging harder
- May add unnecessary complexity



ANTHROPIC'S RECOMMENDATION

Start with direct LLM API calls

Core Terminology: State

State: Current Snapshot

- **Definition:** Current snapshot of your application.
- Represents the "state of the world" at any point.
- Shared across the entire application.
- **Immutable** – never modified, only replaced.



KEY CONCEPT

State is **data**, not a function.

Core Terminology: Graph

What is a Graph?

A tree structure representing your agent workflow

- Nodes connected together
- Hierarchical dependencies
- Visual representation of logic flow



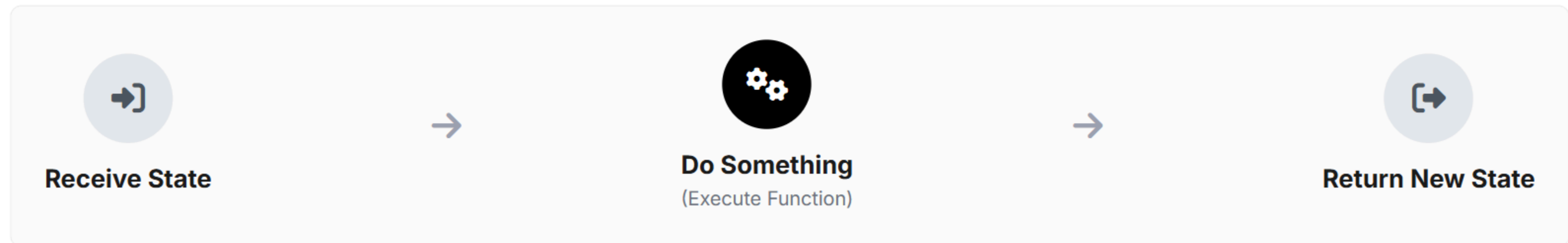
ANALOGY

Think: Flowchart for agent operations

Core Terminology - Nodes

Nodes: Python functions that do work

- Each node represents **one operation**
- **Input:** Current state
- **Process:** Perform logic (call LLM, side effects, etc.)
- **Output:** Return updated state



REMEMBER

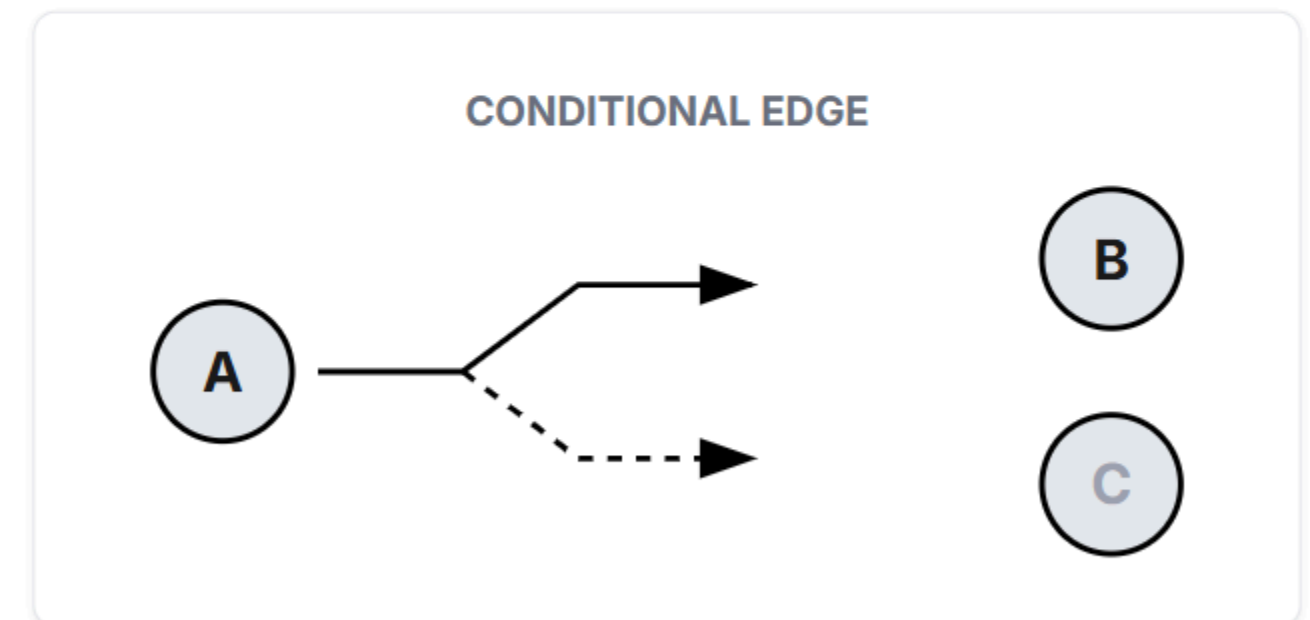
Nodes receive state → do something → return new state

Core Terminology - Edges

↶ What are Edges?

Python functions that determine flow

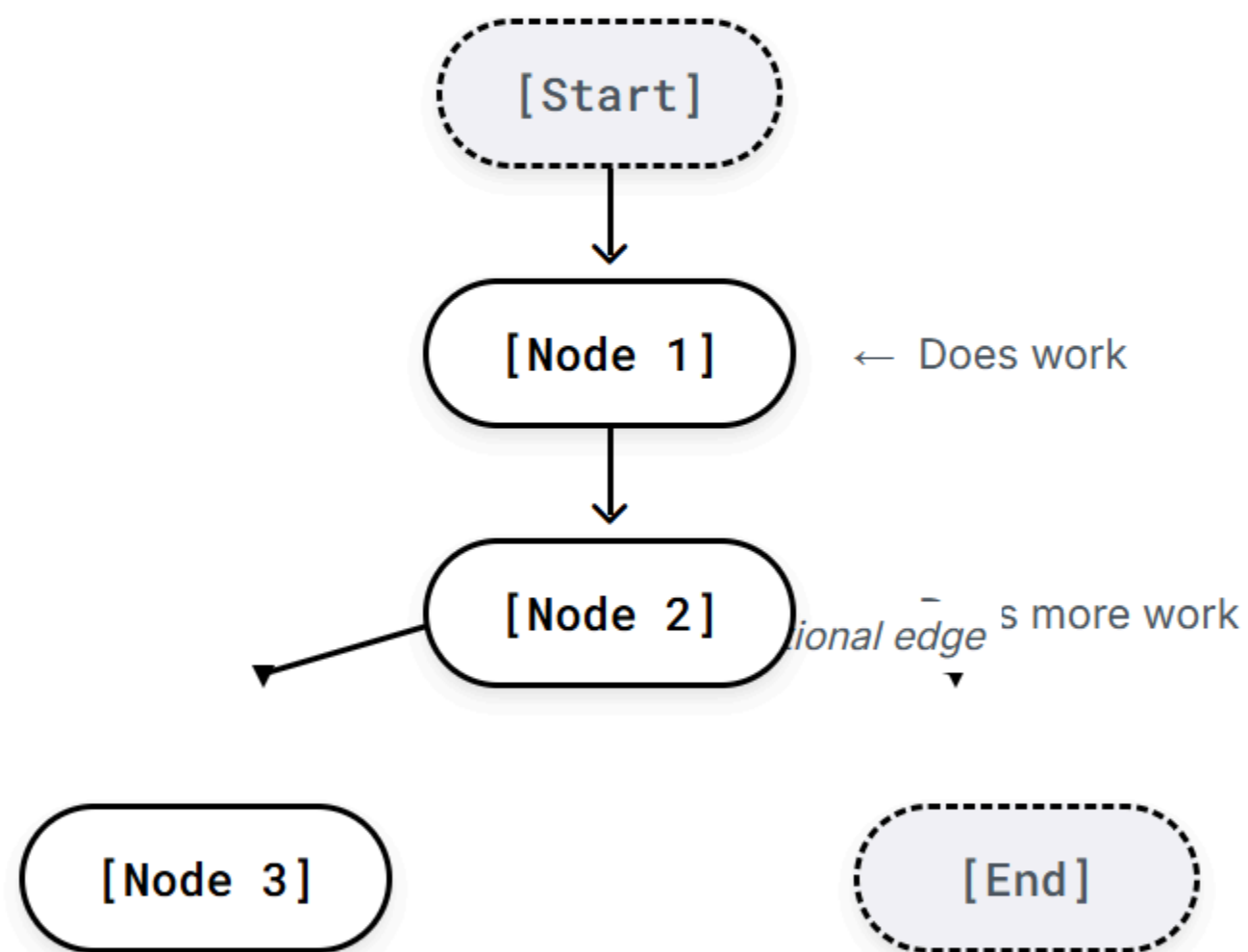
- Connect nodes together
- Decide what happens next based on state
- Two types:
 - **Simple:** Direct connection ($A \rightarrow B$)
 - **Conditional:** Based on conditions (if X then B, else C)



SUMMARY

Nodes do work, edges choose what's next.

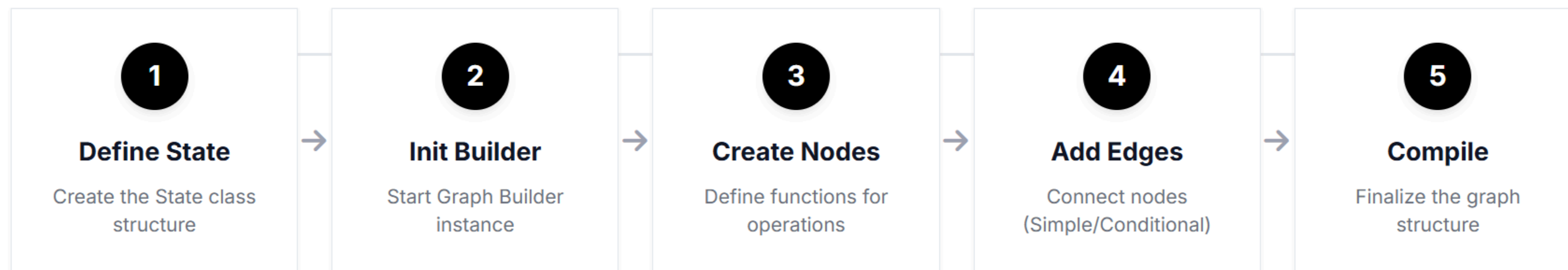
Visual Example



○ **Nodes:** Orange circles (operations) ↓ **Edges:** Arrows (flow control)

Five Steps to Build a Graph

Before your agents run, you must complete the build phase:



EXECUTION PHASE

Then: Invoke the graph to run it

State is Immutable

What "Immutable" Means:

- Never change the state object directly
- Receive old state → return new state
- Maintains snapshots for time travel



EXAMPLE PATTERN

```
def my_node(old_state):  
    # Don't modify old_state  
    new_state = State(updated_value=...)  
  
    return new_state
```

Understanding the Two Phases

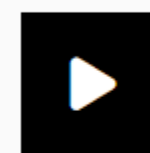


Phase 1: Graph Building

(SETUP)

- Define state structure (schema for the application)
- Create nodes and edges defining operations and flow
- Compile the graph into a runnable executable
- This happens at runtime, before execution

Static Configuration Phase



Phase 2: Graph Execution

(RUNNING)

- Invoke the graph with an initial input state
- Agents perform work within the node functions
- State flows through nodes based on edge logic

Dynamic Runtime Phase

Reducers Explained

Combining State Updates

- **Reducer:** Special function for combining states
- Associated with specific state fields
- Combines new state with current state
- **Why needed?:** Enables parallel node execution



BENEFIT

Multiple nodes can update state simultaneously without conflicts.

Built-in Reducer: add_messages

Most Common Reducer

- Concatenates message lists automatically
- Packages messages properly
(handles `HumanMessage`, `AIMessage` types)
- Manages conversation history seamlessly



USAGE

Annotate state fields with reducers to control updates.

Type Hints and Annotations

Syntax Evolution

- **Standard** Define the expected data type for Type Hints: inputs and outputs.
- **Annotated** Allow you to attach extra metadata or Hints: logic to a type definition.

WHY IT MATTERS

LangGraph uses **annotations** specifically to associate reducer functions with state fields.

1. PYTHON TYPE HINTS

```
def shout(text: str) -> str:  
    return text.upper()
```

2. ANNOTATED TYPE HINTS

```
from typing import Annotated  
  
def shout(text: Annotated[str, "something to shout"]) -> str:  
    return text.upper()
```

Defining State with Pydantic

Breakdown:

- `messages` : Field name
- `list` : Type
- `add_messages` : Reducer function



STATE DEFINITION EXAMPLE

```
from pydantic import BaseModel
from typing import Annotated
from langgraph.graph.message import add_messages

class State(BaseModel):
    messages: Annotated[list, add_messages]
```

Starting the Graph Builder

Step 2 - Initialize Builder

- Import the StateGraph class from the library
- Initialize the builder with your custom state definition

CRITICAL NOTE

Pass the **State class** definition itself, not an instantiated object (instance) of the class.

PYTHON IMPLEMENTATION

```
from langgraph.graph import StateGraph

# Initialize builder with State class
graph_builder = StateGraph(State)
```



Creating a Node

Step 3 - Define Node Function

Nodes are simply Python functions that process state.



IMPLEMENTATION

```
def my_node(old_state: State) -> State:
    # Do something
    result = some_operation()

    # Return new state
    return State(messages=[result])

# Add to graph
graph_builder.add_node("my_node", my_node)
```

Compiling and Visualizing

Step 5 - Compile

- Benefit: See your workflow visually before running
- Turns definition into executable
- Allows inspection of graph structure



COMPILATION CODE

```
graph = graph_builder.compile()

# Visualize the graph
from IPython.display import Image, display

display(Image(
    graph.get_graph().draw_mermaid_png()
))
```

Creating Edges

Step 4 - Connect Nodes:

- Connects nodes to define execution flow
- **START** and **END** are special constants provided by the library



EDGE DEFINITION

```
from langgraph.graph import START, END

# Simple edges
graph_builder.add_edge(START, "my_node")
graph_builder.add_edge("my_node", END)
```


Running the Graph

Invoke to Execute

KEY CONCEPT

The **`graph.invoke(state)`** method is the trigger that executes the entire workflow, passing the initial state through your nodes.



EXECUTION SCRIPT

```
initial_state = State(  
    messages=[{"role": "user", "content": "Hello"}]  
)  
  
# Run the graph with input state  
result = graph.invoke(initial_state)  
  
print(result)
```

Simple Example - Random Responses



Demo Purpose: Show graphs work without LLMs

- Node picks random noun + adjective
- Returns silly sentences
- Demonstrates state flow



TAKEAWAY

Nodes are just Python functions (any logic works)

Integration with LangChain

⚙️ Optional but Convenient

- LangChain provides `ChatOpenAI` wrapper
- Simplifies LLM calls
- Community examples use it
- **Alternative:** Use OpenAI SDK directly



FLEXIBILITY

LangGraph works with any framework.

Real Example - Chatbot Node

Using LangChain's ChatOpenAI



PYTHON IMPLEMENTATION

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini")

def chatbot_node(old_state: State) -> State:
    response = llm.invoke(old_state.messages)
    return State(messages=[response])
```

Current Limitation: No Memory

⚠ Problem: Each invoke is independent

- No conversation history maintained between calls
- Graph doesn't remember previous messages
- Each call starts fresh (Stateless execution)



COMING NEXT

Solution: Coming in Day 22 (tomorrow)!

What's Next - Day 22

Tomorrow We'll Add

- Conversation memory/history
- Tool calling capabilities
- Conditional edges
- More complex workflows



GET READY

Building on today's foundation

Resources



LangGraph Documentation

<https://langchain-ai.github.io/langgraph/>



LangChain Documentation

<https://python.langchain.com/>



LangSmith

<https://smith.langchain.com/>



Anthropic Blog: Building Effective Agents

<https://www.anthropic.com/research/building-effective-agents>



LangGraph GitHub Repository

<https://github.com/langchain-ai/langgraph>



Key Takeaways

LangGraph

01

Framework for robust agent workflows focused on stability and resiliency.

Graph

02

Tree structure of nodes and edges representing logical dependencies.

State

03

Immutable snapshot of the application "world" shared across the graph.

Nodes

04

Python functions that perform work and return an updated state.

Edges

05

Control flow functions (simple or conditional) determining the next step.

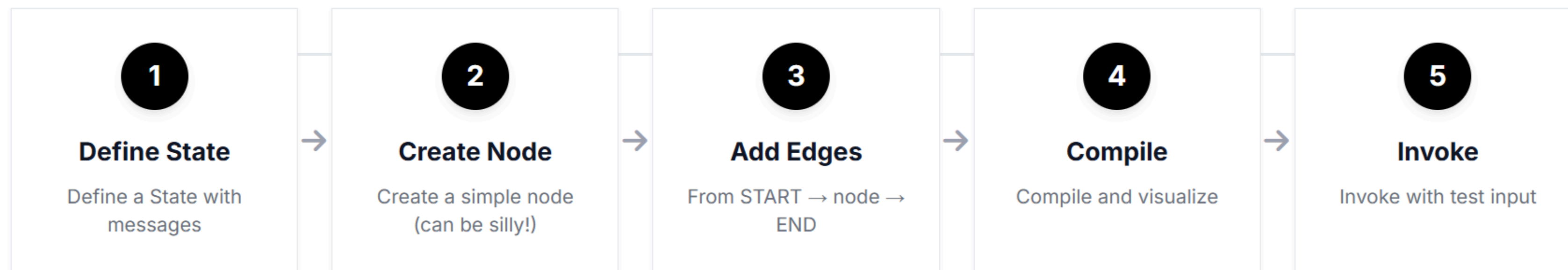
Two Phases

06

Build Graph (Setup) → **Execute Graph** (Runtime)

Practice Challenge

Build Your First Graph:



OBJECTIVE

Goal: Get comfortable with the five-step process

Questions to Consider



How does LangGraph compare to previous frameworks?



When would you choose LangGraph over others?



What problems does the graph structure solve?



How does immutability help with reliability?



Thank You!

✓ Day 21 Complete

- Learned LangGraph fundamentals
- Understood graphs, states, nodes, and edges
- Built first simple graph
- Ready for advanced features tomorrow



COMING UP NEXT

See you in Day 22!